# Library Energy Usage During Exams

## Objective

Analyze and forecast library energy consumption during exam periods by:

1. Aggregating historical energy usage with academic event calendars
2. Implementing Exponential Smoothing (Holt-Winters) for semester-end forecasts
3. Creating a Streamlit dashboard with gauge visualization

## Approach

- Generate synthetic historical library energy data with exam period patterns
- Merge energy data with academic calendar events
- Apply Triple Exponential Smoothing for seasonal forecasting
- Build an interactive Streamlit dashboard with gauge display

# Step 1: Install Required Libraries

```
%pip install pandas numpy matplotlib plotly statsmodels streamlit plotly-exp
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.1/9.1 MB 51.5 MB/s eta 0:00:00
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 6.9/6.9 MB 45.5 MB/s eta 0:00:00
```

# Step 2: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
import plotly.graph_objects as go
from datetime import datetime, timedelta
from statsmodels.tsa.holtwinters import ExponentialSmoothing
import warnings
warnings.filterwarnings('ignore')

print(" Libraries imported successfully!")
```

```
 Libraries imported successfully!
```

# Step 3: Generate Synthetic Library Energy Data with Academic Calendar

We'll create 2 years of daily library energy consumption data that reflects:

- **Weekday patterns**: Higher usage on weekdays
- **Exam periods**: Significant spike during mid-term and final exams
- **Seasonal variations**: Winter/summer semester differences
- **Holiday dips**: Lower usage during breaks

```
np.random.seed(42)

# Generate date range (2 years of data)
start_date = datetime(2024, 1, 1)
end_date = datetime(2025, 12, 31)
dates = pd.date_range(start=start_date, end=end_date, freq='D')

# Base energy consumption (kWh)
base_consumption = 450

# Create academic calendar events
def create_academic_calendar():
    """Create academic event calendar with exam periods and holidays"""
    events = []

    for year in [2024, 2025]:
        # Spring semester mid-terms (March)
        for day in range(10, 22):
            events.append({'date': datetime(year, 3, day), 'event': 'Midterm

        # Spring semester finals (May)
        for day in range(5, 20):
            events.append({'date': datetime(year, 5, day), 'event': 'Final E

        # Fall semester mid-terms (October)
        for day in range(15, 28):
            events.append({'date': datetime(year, 10, day), 'event': 'Midter

        # Fall semester finals (December)
        for day in range(1, 15):
            events.append({'date': datetime(year, 12, day), 'event': 'Final

        # Winter break (late December - early January)
        for day in range(20, 32):
            try:
                events.append({'date': datetime(year, 12, day), 'event': 'Wi
            except:
                pass
        for day in range(1, 10):
            events.append({'date': datetime(year, 1, day), 'event': 'Winter
```

```
        # Summer break (June-July)
        for month in [6, 7]:
            for day in range(1, 29):
                try:
                    events.append({'date': datetime(year, month, day), 'even
                except:
                    pass

        # Regular semester weeks
        # Spring regular (February, April)
        for month in [2, 4]:
            for day in range(1, 29):
                try:
                    events.append({'date': datetime(year, month, day), 'even
                except:
                    pass

        # Fall regular (September, November)
        for month in [9, 11]:
            for day in range(1, 31):
                try:
                    events.append({'date': datetime(year, month, day), 'even
                except:
                    pass

    return pd.DataFrame(events)

# Create calendar
calendar_df = create_academic_calendar()
calendar_df['date'] = pd.to_datetime(calendar_df['date'])
calendar_df = calendar_df.drop_duplicates(subset='date', keep='first')

print(f" Academic Calendar created with {len(calendar_df)} event days")
calendar_df.groupby('event')['multiplier'].agg(['count', 'mean'])
```

```
 Academic Calendar created with 494 event days
```

| event | count | mean |
|---|---|---|
| Final Exams | 58 | 1.8 |
| Midterm Exams | 50 | 1.6 |
| Regular Semester | 232 | 1.0 |
| Summer Break | 112 | 0.5 |
| Winter Break | 42 | 0.3 |

```
# Generate library energy consumption data
def generate_energy_data(dates, calendar_df, base_consumption):
    """Generate synthetic library energy consumption with realistic patterns

    data = []
```

```
    for date in dates:
        consumption = base_consumption

        # Day of week effect (weekends lower)
        day_of_week = date.dayofweek
        if day_of_week == 5:  # Saturday
            consumption *= 0.6
        elif day_of_week == 6:  # Sunday
            consumption *= 0.4

        # Seasonal effect (heating/cooling)
        month = date.month
        if month in [12, 1, 2]:  # Winter - heating
            consumption *= 1.25
        elif month in [6, 7, 8]:  # Summer - cooling
            consumption *= 1.15

        # Apply academic calendar multiplier
        calendar_match = calendar_df[calendar_df['date'] == date]
        if len(calendar_match) > 0:
            event = calendar_match.iloc[0]['event']
            multiplier = calendar_match.iloc[0]['multiplier']
            consumption *= multiplier
        else:
            event = 'Regular'
            multiplier = 1.0

        # Add random noise (±10%)
        noise = np.random.uniform(-0.10, 0.10)
        consumption *= (1 + noise)

        # Extended hours during exams (additional lighting/HVAC)
        if 'Exam' in str(event):
            # Libraries open longer during exams
            consumption += np.random.uniform(50, 100)

        data.append({
            'date': date,
            'energy_kwh': round(consumption, 2),
            'event': event,
            'day_of_week': date.strftime('%A'),
            'month': date.strftime('%B'),
            'is_weekend': day_of_week >= 5,
            'is_exam_period': 'Exam' in str(event)
        })

    return pd.DataFrame(data)

# Generate the energy dataset
energy_df = generate_energy_data(dates, calendar_df, base_consumption)

print(f" Generated {len(energy_df)} days of library energy data")
print(f" Date range: {energy_df['date'].min().date()} to {energy_df['date'].
print(f"\n Energy Statistics:")
energy_df['energy_kwh'].describe()
```

```
Generated 731 days of library energy data
Date range: 2024-01-01 to 2025-12-31

Energy Statistics:
```

| | energy_kwh |
|---|---|
| **count** | 731.000000 |
| **mean** | 425.370000 |
| **std** | 219.091662 |
| **min** | 61.530000 |
| **25%** | 250.725000 |
| **50%** | 434.070000 |
| **75%** | 517.435000 |
| **max** | 1202.630000 |

**dtype:** float64

## Step 4: Aggregate Historical Usage with Event Calendar

```
# Aggregate energy by event type
event_aggregation = energy_df.groupby('event').agg({
    'energy_kwh': ['mean', 'sum', 'std', 'count'],
    'is_exam_period': 'first'
}).round(2)

event_aggregation.columns = ['Avg Daily (kWh)', 'Total (kWh)', 'Std Dev', 'D
event_aggregation = event_aggregation.sort_values('Avg Daily (kWh)', ascendi

print(" Energy Consumption by Academic Event Type:")
print("=" * 70)
event_aggregation
```

```
Energy Consumption by Academic Event Type:
======================================================================
```

| event | Avg Daily (kWh) | Total (kWh) | Std Dev | Days | Is Exam |
|---|---|---|---|---|---|
| **Final Exams** | 855.93 | 49643.93 | 246.17 | 58 | True |
| **Midterm Exams** | 702.53 | 35126.74 | 168.55 | 50 | True |
| **Regular** | 425.30 | 100796.82 | 122.26 | 237 | False |
| **Regular Semester** | 407.87 | 94625.81 | 123.18 | 232 | False |
| **Summer Break** | 219.95 | 24634.41 | 60.78 | 112 | False |
| **Winter Break** | 145.66 | 6117.76 | 42.85 | 42 | False |

Next steps: Generate code with `event_aggregation` New interactive sheet

```
# Monthly aggregation with exam period identification
monthly_agg = energy_df.groupby([energy_df['date'].dt.to_period('M')]).agg({
    'energy_kwh': ['sum', 'mean', 'max'],
    'is_exam_period': 'sum'
}).round(2)

monthly_agg.columns = ['Total kWh', 'Avg Daily kWh', 'Peak kWh', 'Exam Days'
monthly_agg.index = monthly_agg.index.astype(str)

print("\n Monthly Energy Aggregation with Exam Days:")
monthly_agg
```

```
Monthly Energy Aggregation with Exam Days:
```

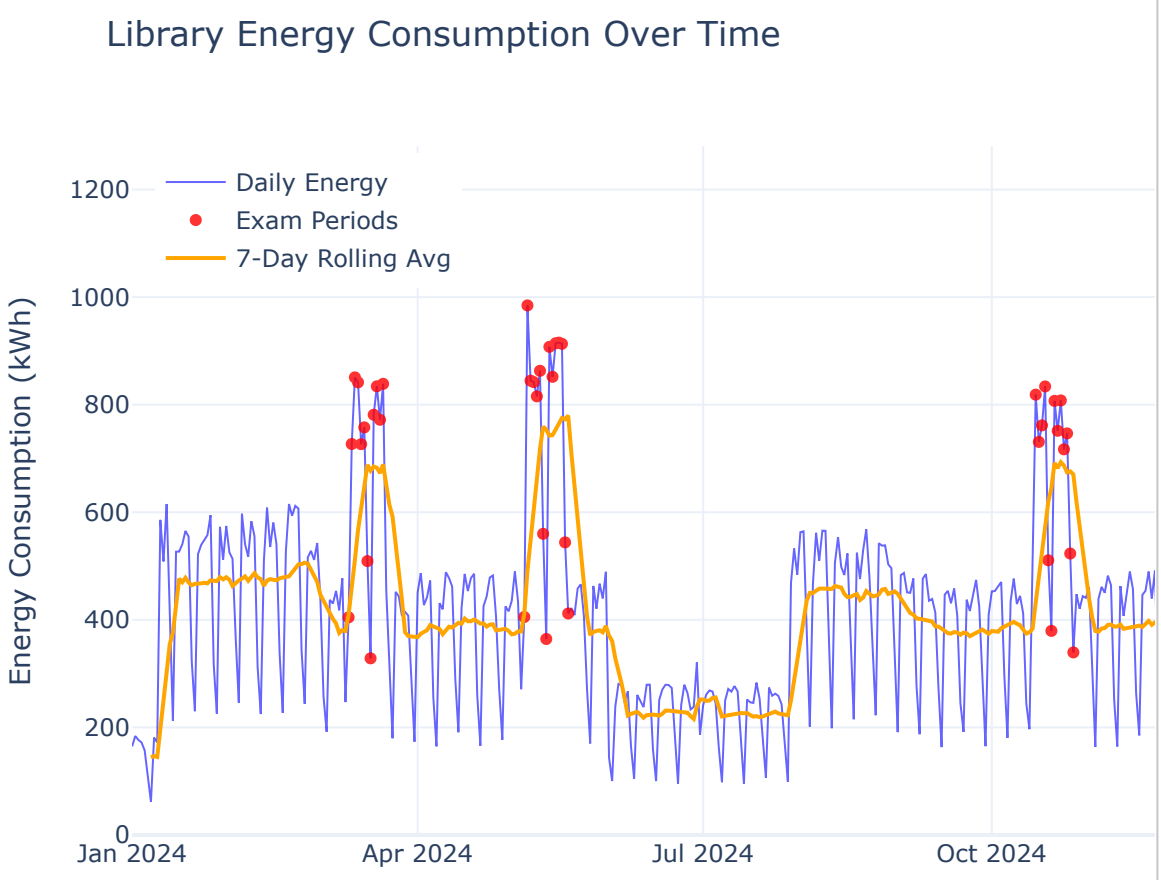
| date | Total kWh | Avg Daily kWh | Peak kWh | Exam Days |
|---|---|---|---|---|
| **2024-01** | 11875.36 | 383.08 | 615.36 | 0 |
| **2024-02** | 13967.03 | 481.62 | 615.33 | 0 |
| **2024-03** | 15220.68 | 490.99 | 850.77 | 12 |
| **2024-04** | 11734.11 | 391.14 | 488.67 | 0 |
| **2024-05** | 17649.45 | 569.34 | 984.55 | 15 |
| **2024-06** | 6757.42 | 225.25 | 320.65 | 0 |
| **2024-07** | 7723.12 | 249.13 | 532.56 | 0 |
| **2024-08** | 14044.54 | 453.05 | 568.22 | 0 |
| **2024-09** | 11381.71 | 379.39 | 487.19 | 0 |
| **2024-10** | 15914.23 | 513.36 | 833.91 | 13 |
| **2024-11** | 11478.89 | 382.63 | 491.51 | 0 |
| **2024-12** | 17654.71 | 569.51 | 1192.65 | 14 |
| **2025-01** | 11943.86 | 385.29 | 617.25 | 0 |
| **2025-02** | 13557.50 | 484.20 | 617.96 | 0 |
| **2025-03** | 15854.45 | 511.43 | 879.37 | 12 |
| **2025-04** | 11972.74 | 399.09 | 494.97 | 0 |
| **2025-05** | 17582.15 | 567.17 | 966.18 | 15 |
| **2025-06** | 6788.22 | 226.27 | 543.11 | 0 |
| **2025-07** | 7637.33 | 246.37 | 532.60 | 0 |
| **2025-08** | 13374.35 | 431.43 | 566.09 | 0 |
| **2025-09** | 11592.26 | 386.41 | 494.10 | 0 |
| **2025-10** | 16068.43 | 518.34 | 866.15 | 13 |
| **2025-11** | 11258.71 | 375.29 | 490.26 | 0 |
| **2025-12** | 17914.22 | 577.88 | 1202.63 | 14 |

Next steps: Generate code with `monthly_agg` New interactive sheet

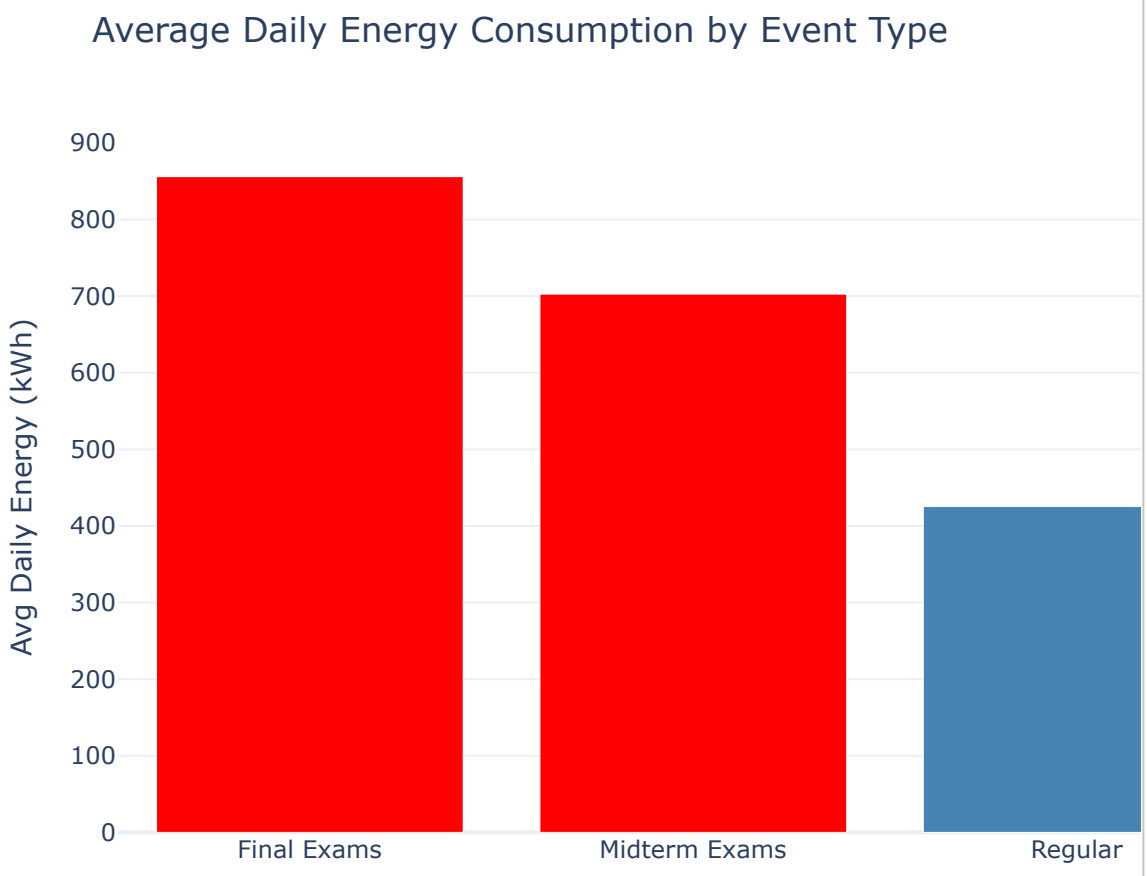
## Step 5: Visualize Historical Energy Patterns

```
# Time series plot with exam periods highlighted
fig = go.Figure()
```

```
# Main energy consumption line
fig.add_trace(go.Scatter(
    x=energy_df['date'],
    y=energy_df['energy_kwh'],
    mode='lines',
    name='Daily Energy',
    line=dict(color='blue', width=1),
    opacity=0.6
))

# Highlight exam periods
exam_data = energy_df[energy_df['is_exam_period']]
fig.add_trace(go.Scatter(
    x=exam_data['date'],
    y=exam_data['energy_kwh'],
    mode='markers',
    name='Exam Periods',
    marker=dict(color='red', size=6, symbol='circle'),
    opacity=0.8
))

# Add 7-day rolling average
energy_df['rolling_avg'] = energy_df['energy_kwh'].rolling(window=7).mean()
fig.add_trace(go.Scatter(
    x=energy_df['date'],
    y=energy_df['rolling_avg'],
    mode='lines',
    name='7-Day Rolling Avg',
    line=dict(color='orange', width=2)
))

fig.update_layout(
    title=' Library Energy Consumption Over Time',
    xaxis_title='Date',
    yaxis_title='Energy Consumption (kWh)',
    template='plotly_white',
    hovermode='x unified',
    legend=dict(yanchor="top", y=0.99, xanchor="left", x=0.01)
)

fig.show()
```




```
# Energy consumption by event type - Bar chart
fig_bar = px.bar(
    event_aggregation.reset_index(),
    x='event',
    y='Avg Daily (kWh)',
    color='Is Exam',
    color_discrete_map={True: 'red', False: 'steelblue'},
    title=' Average Daily Energy Consumption by Event Type',
    labels={'event': 'Academic Event', 'Avg Daily (kWh)': 'Avg Daily Energy
)

fig_bar.update_layout(template='plotly_white', showlegend=True)
fig_bar.show()
```




## Step 6: Implement Exponential Smoothing (Holt-Winters)

Triple Exponential Smoothing (Holt-Winters) captures:

- **Level**: The baseline value of the series
- **Trend**: The increasing/decreasing tendency
- **Seasonality**: Recurring patterns (weekly in our case)

```
# Prepare data for exponential smoothing
# Resample to weekly for better seasonality detection
weekly_energy = energy_df.set_index('date')['energy_kwh'].resample('W').mean

# Split data: training (first 80%) and test (last 20%)
train_size = int(len(weekly_energy) * 0.8)
train_data = weekly_energy[:train_size]
test_data = weekly_energy[train_size:]

print(f" Training data: {len(train_data)} weeks")
print(f" Test data: {len(test_data)} weeks")
print(f" Training period: {train_data.index[0].date()} to {train_data.index[
print(f" Test period: {test_data.index[0].date()} to {test_data.index[-1].da
```

```
 Training data: 84 weeks
 Test data: 21 weeks
 Training period: 2024-01-07 to 2025-08-10
 Test period: 2025-08-17 to 2026-01-04
```

```
# Fit Holt-Winters Exponential Smoothing model
# Using multiplicative seasonality with 52-week (annual) seasonal period
# We'll use a shorter period of 13 weeks (quarterly) for better fit

model = ExponentialSmoothing(
    train_data,
    seasonal_periods=13,  # Quarterly seasonality (semester cycles)
    trend='add',          # Additive trend
    seasonal='mul',       # Multiplicative seasonality
    damped_trend=True     # Damped trend for better long-term forecasts
)

# Fit the model
fitted_model = model.fit(optimized=True)

# Display model parameters
print(" Holt-Winters Model Parameters:")
print("=" * 50)
print(f"Alpha (Level):      {fitted_model.params['smoothing_level']:.4f}")
print(f"Beta (Trend):       {fitted_model.params['smoothing_trend']:.4f}")
print(f"Gamma (Seasonal):   {fitted_model.params['smoothing_seasonal']:.4f}"
print(f"Phi (Damping):      {fitted_model.params['damping_trend']:.4f}")
```

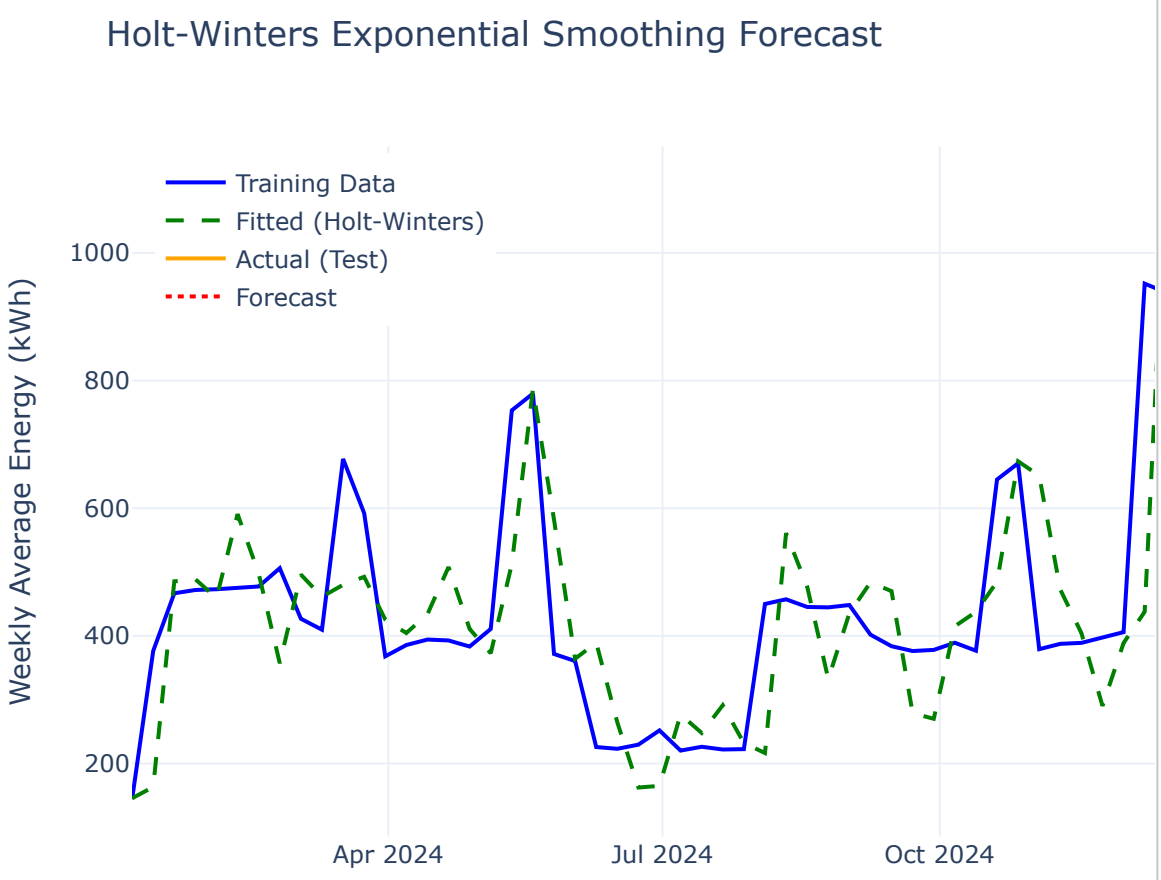
```
 Holt-Winters Model Parameters:
==================================================
Alpha (Level):      1.0000
Beta (Trend):       0.0000
Gamma (Seasonal):   0.0000
Phi (Damping):      0.9547
```

```
# Generate forecasts for semester-end period
forecast_periods = len(test_data) + 8  # Forecast through test period + 2 mo
forecast = fitted_model.forecast(steps=forecast_periods)

# Calculate fitted values
fitted_values = fitted_model.fittedvalues

# Evaluate model on test data
test_forecast = forecast[:len(test_data)]
mae = np.mean(np.abs(test_data.values - test_forecast.values))
rmse = np.sqrt(np.mean((test_data.values - test_forecast.values) ** 2))
mape = np.mean(np.abs((test_data.values - test_forecast.values) / test_data.

print("\n Forecast Accuracy Metrics:")
print("=" * 50)
print(f"Mean Absolute Error (MAE):        {mae:.2f} kWh")
print(f"Root Mean Square Error (RMSE):    {rmse:.2f} kWh")
print(f"Mean Absolute % Error (MAPE):     {mape:.2f}%")
```

```
 Forecast Accuracy Metrics:
==================================================
Mean Absolute Error (MAE):        152.22 kWh
Root Mean Square Error (RMSE):    221.49 kWh
Mean Absolute % Error (MAPE):     29.55%
```

```
# Visualize forecast results
fig_forecast = go.Figure()

# Training data
fig_forecast.add_trace(go.Scatter(
    x=train_data.index,
    y=train_data.values,
    mode='lines',
    name='Training Data',
    line=dict(color='blue', width=2)
))

# Fitted values
fig_forecast.add_trace(go.Scatter(
    x=fitted_values.index,
    y=fitted_values.values,
    mode='lines',
    name='Fitted (Holt-Winters)',
    line=dict(color='green', width=2, dash='dash')
))

# Test data (actual)
fig_forecast.add_trace(go.Scatter(
    x=test_data.index,
    y=test_data.values,
    mode='lines',
    name='Actual (Test)',
    line=dict(color='orange', width=2)
))

# Forecast
fig_forecast.add_trace(go.Scatter(
    x=forecast.index,
    y=forecast.values,
    mode='lines',
    name='Forecast',
    line=dict(color='red', width=2, dash='dot')
))

# Add vertical line at forecast start using add_shape
forecast_start = test_data.index[0]
fig_forecast.add_shape(
    type="line",
    x0=forecast_start, x1=forecast_start,
    y0=0, y1=1,
    yref="paper",
    line=dict(color="gray", width=2, dash="dash")
```

```
)

# Add annotation for the vertical line
fig_forecast.add_annotation(
    x=forecast_start,
    y=1,
    yref="paper",
    text="Forecast Start",
    showarrow=False,
    font=dict(color="gray"),
    yshift=10
)

fig_forecast.update_layout(
    title=' Holt-Winters Exponential Smoothing Forecast',
    xaxis_title='Date',
    yaxis_title='Weekly Average Energy (kWh)',
    template='plotly_white',
    hovermode='x unified',
    legend=dict(yanchor="top", y=0.99, xanchor="left", x=0.01)
)

fig_forecast.show()
```



## Step 7: Semester-End Forecast Summary

Calculate key metrics for the upcoming semester-end (exam period) forecast.

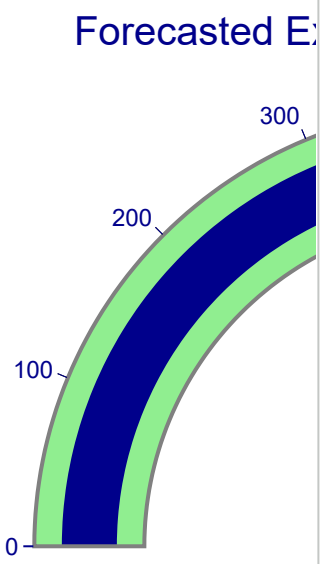
```
# Calculate semester-end forecast metrics
forecast_df = pd.DataFrame({
    'date': forecast.index,
    'forecasted_kwh': forecast.values
})

# Next semester-end forecast (next 4 weeks represent exam period)
next_exam_forecast = forecast[-8:-4].mean()  # 4 weeks of exam period
current_avg = weekly_energy[-13:].mean()  # Last quarter average
historical_exam_avg = energy_df[energy_df['is_exam_period']]['energy_kwh'].m

# Calculate percentage of capacity (assume max capacity is 800 kWh/day equiv
max_capacity = 800
capacity_utilization = (next_exam_forecast / max_capacity) * 100

print(" Semester-End Forecast Summary")
print("=" * 60)
print(f" Forecast Period: Next Exam Season")
print(f" Forecasted Weekly Avg:      {next_exam_forecast:.2f} kWh")
print(f" Current Quarterly Avg:      {current_avg:.2f} kWh")
print(f" Historical Exam Avg:        {historical_exam_avg:.2f} kWh")
print(f" Expected Change:            {((next_exam_forecast - current_avg) /
print(f" Capacity Utilization:       {capacity_utilization:.1f}%")

# Store for dashboard
forecast_metrics = {
    'forecast_value': next_exam_forecast,
    'current_value': current_avg,
    'historical_exam': historical_exam_avg,
    'capacity_pct': capacity_utilization,
    'max_capacity': max_capacity
}
```

```
 Semester-End Forecast Summary
============================================================
 Forecast Period: Next Exam Season
 Forecasted Weekly Avg:      320.66 kWh
 Current Quarterly Avg:      485.11 kWh
 Historical Exam Avg:        784.91 kWh
 Expected Change:            -33.9%
 Capacity Utilization:       40.1%
```

## Step 8: Create Gauge Visualization (Dashboard Preview)

```
# Create gauge chart for forecasted energy consumption
fig_gauge = go.Figure(go.Indicator(
    mode="gauge+number+delta",
    value=next_exam_forecast,
```

```
        domain={'x': [0, 1], 'y': [0, 1]},
        title={'text': "Forecasted Exam Period Energy (kWh/week)", 'font': {'siz
        delta={'reference': current_avg, 'increasing': {'color': "red"}, 'decrea
        gauge={
            'axis': {'range': [None, max_capacity], 'tickwidth': 1, 'tickcolor':
            'bar': {'color': "darkblue"},
            'bgcolor': "white",
            'borderwidth': 2,
            'bordercolor': "gray",
            'steps': [
                {'range': [0, max_capacity * 0.5], 'color': 'lightgreen'},
                {'range': [max_capacity * 0.5, max_capacity * 0.75], 'color': 'y
                {'range': [max_capacity * 0.75, max_capacity], 'color': 'salmon'
            ],
            'threshold': {
                'line': {'color': "red", 'width': 4},
                'thickness': 0.75,
                'value': max_capacity * 0.9
            }
        }
    ))

    fig_gauge.update_layout(
        paper_bgcolor="white",
        font={'color': "darkblue", 'family': "Arial"},
        height=400
    )

    fig_gauge.show()

    print("\n Gauge Legend:")
    print("   Green Zone (0-50%):    Normal consumption")
    print("   Yellow Zone (50-75%):  Elevated consumption")
    print("   Red Zone (75-100%):    High consumption - monitor closely")
```




```
Gauge Legend:
  Green Zone (0-50%):    Normal consumption
  Yellow Zone (50-75%):  Elevated consumption
  Red Zone (75-100%):    High consumption - monitor closely
```

## Step 9: Generate Streamlit Dashboard Code

The following cell creates a complete Streamlit dashboard file that can be run independently.

```
streamlit_code = '''
import streamlit as st
import pandas as pd
import numpy as np
import plotly.graph_objects as go
import plotly.express as px
from datetime import datetime, timedelta
from statsmodels.tsa.holtwinters import ExponentialSmoothing
import warnings
warnings.filterwarnings('ignore')

# Page configuration
st.set_page_config(
    page_title="Library Energy Dashboard",
    page_icon="📚",
    layout="wide"
)

# Title
st.title(" Library Energy During Exams Dashboard")
```

```
st.markdown("**Aggregated Historical Usage with Exponential Smoothing Forec

# Sidebar
st.sidebar.header("Settings")
forecast_weeks = st.sidebar.slider("Forecast Horizon (weeks)", 4, 16, 8)
seasonal_period = st.sidebar.selectbox("Seasonal Period", [13, 26, 52], inc
show_confidence = st.sidebar.checkbox("Show Historical Comparison", True)

# Generate synthetic data (same as notebook)
@st.cache_data
def generate_data():
    np.random.seed(42)
    start_date = datetime(2024, 1, 1)
    end_date = datetime(2025, 12, 31)
    dates = pd.date_range(start=start_date, end=end_date, freq='D')
    base_consumption = 450

    # Create academic calendar
    events = []
    for year in [2024, 2025]:
        for day in range(10, 22):
            events.append({'date': datetime(year, 3, day), 'event': 'Midter
        for day in range(5, 20):
            events.append({'date': datetime(year, 5, day), 'event': 'Final
        for day in range(15, 28):
            events.append({'date': datetime(year, 10, day), 'event': 'Midte
        for day in range(1, 15):
            events.append({'date': datetime(year, 12, day), 'event': 'Final
        for day in range(20, 32):
            try:
                events.append({'date': datetime(year, 12, day), 'event': 'W
            except:
                pass
        for day in range(1, 10):
            events.append({'date': datetime(year, 1, day), 'event': 'Winter
        for month in [6, 7]:
            for day in range(1, 29):
                try:
                    events.append({'date': datetime(year, month, day), 'eve
                except:
                    pass

    calendar_df = pd.DataFrame(events)
    calendar_df['date'] = pd.to_datetime(calendar_df['date'])
    calendar_df = calendar_df.drop_duplicates(subset='date', keep='first')

    data = []
    for date in dates:
        consumption = base_consumption
        day_of_week = date.dayofweek
        if day_of_week == 5:
            consumption *= 0.6
        elif day_of_week == 6:
            consumption *= 0.4
        month = date.month
```

```
            if month in [12, 1, 2]:
                consumption *= 1.25
            elif month in [6, 7, 8]:
                consumption *= 1.15

            calendar_match = calendar_df[calendar_df['date'] == date]
            if len(calendar_match) > 0:
                event = calendar_match.iloc[0]['event']
                multiplier = calendar_match.iloc[0]['multiplier']
                consumption *= multiplier
            else:
                event = 'Regular'

            noise = np.random.uniform(-0.10, 0.10)
            consumption *= (1 + noise)

            if 'Exam' in str(event):
                consumption += np.random.uniform(50, 100)

            data.append({
                'date': date,
                'energy_kwh': round(consumption, 2),
                'event': event,
                'is_exam_period': 'Exam' in str(event)
            })

    return pd.DataFrame(data)

# Load data
energy_df = generate_data()
weekly_energy = energy_df.set_index('date')['energy_kwh'].resample('W').mea

# Fit model
@st.cache_resource
def fit_model(data, seasonal):
    model = ExponentialSmoothing(
        data,
        seasonal_periods=seasonal,
        trend='add',
        seasonal='mul',
        damped_trend=True
    )
    return model.fit(optimized=True)

fitted_model = fit_model(weekly_energy, seasonal_period)
forecast = fitted_model.forecast(steps=forecast_weeks)

# Calculate metrics
max_capacity = 800
next_exam_forecast = forecast.mean()
current_avg = weekly_energy[-13:].mean()
historical_exam_avg = energy_df[energy_df['is_exam_period']]['energy_kwh'].
capacity_utilization = (next_exam_forecast / max_capacity) * 100

# Layout
```

```
col1, col2, col3, col4 = st.columns(4)

with col1:
    st.metric(
        label="Forecasted Weekly Avg",
        value=f"{next_exam_forecast:.1f} kWh",
        delta=f"{((next_exam_forecast - current_avg) / current_avg * 100):+
    )

with col2:
    st.metric(
        label="Current Avg",
        value=f"{current_avg:.1f} kWh"
    )

with col3:
    st.metric(
        label="Historical Exam Avg",
        value=f"{historical_exam_avg:.1f} kWh"
    )

with col4:
    st.metric(
        label="Capacity Utilization",
        value=f"{capacity_utilization:.1f}%"
    )

st.divider()

# Main content
left_col, right_col = st.columns([2, 1])

with left_col:
    st.subheader("Energy Forecast Timeline")

    fig_forecast = go.Figure()

    fig_forecast.add_trace(go.Scatter(
        x=weekly_energy.index,
        y=weekly_energy.values,
        mode='lines',
        name='Historical',
        line=dict(color='blue', width=2)
    ))

    fig_forecast.add_trace(go.Scatter(
        x=forecast.index,
        y=forecast.values,
        mode='lines',
        name='Forecast',
        line=dict(color='red', width=2, dash='dot')
    ))

    fig_forecast.update_layout(
        xaxis_title='Date',
```

```
            yaxis_title='Weekly Avg Energy (kWh)',
            template='plotly_white',
            height=400
        )

        st.plotly_chart(fig_forecast, use_container_width=True)

    with right_col:
        st.subheader("Semester-End Forecast")

        fig_gauge = go.Figure(go.Indicator(
            mode="gauge+number+delta",
            value=next_exam_forecast,
            domain={'x': [0, 1], 'y': [0, 1]},
            title={'text': "Energy (kWh/week)"},
            delta={'reference': current_avg, 'increasing': {'color': "red"}},
            gauge={
                'axis': {'range': [None, max_capacity]},
                'bar': {'color': "darkblue"},
                'steps': [
                    {'range': [0, max_capacity * 0.5], 'color': 'lightgreen'},
                    {'range': [max_capacity * 0.5, max_capacity * 0.75], 'color
                    {'range': [max_capacity * 0.75, max_capacity], 'color': 'sa
                ],
                'threshold': {
                    'line': {'color': "red", 'width': 4},
                    'thickness': 0.75,
                    'value': max_capacity * 0.9
                }
            }
        ))

        fig_gauge.update_layout(height=350)
        st.plotly_chart(fig_gauge, use_container_width=True)

    st.divider()

    # Event comparison
    if show_confidence:
        st.subheader("Energy by Academic Event")
        event_agg = energy_df.groupby('event')['energy_kwh'].mean().sort_values

        fig_bar = px.bar(
            x=event_agg.index,
            y=event_agg.values,
            color=event_agg.values,
            color_continuous_scale='RdYlGn_r',
            labels={'x': 'Event Type', 'y': 'Avg Daily Energy (kWh)'}
        )
        fig_bar.update_layout(template='plotly_white', height=300, showlegend=F
        st.plotly_chart(fig_bar, use_container_width=True)

    # Footer
    st.divider()
    st.caption("Library Energy Analysis Dashboard | Powered by Holt-Winters Exp
```

```
'''

# Save the Streamlit app with UTF-8 encoding
with open('library_energy_dashboard.py', 'w', encoding='utf-8') as f:
    f.write(streamlit_code)

print(" Streamlit dashboard saved as 'library_energy_dashboard.py'")
print("\n To run the dashboard, execute:")
```

```
 Streamlit dashboard saved as 'library_energy_dashboard.py'

 To run the dashboard, execute:
   streamlit run library_energy_dashboard.py
```

## Step 10: Save Data for Dashboard